TECHNICAL ARCHITECTURE & PROOF OF CONCEPT BLUEPRINT

Autonomous Multi-Agent Insurance Claims Processing

Core Architecture: Multi-Agent State Graph + Agentic RAG + MCP

Pipeline: Intake → Policy → Fraud → Risk → Settlement

1. Executive Summary: Connecting Multi-Agent, RAG & MCP

Traditional insurance claims processing suffers from fragmented handoffs, manual document reviews, and delayed settlements. This Proof of Concept (PoC) connects five specialized AI agents—**Claims Intake**, **Policy Validation**, **Fraud Detection**, **Risk Assessment**, and **Settlement**—into a unified, autonomous pipeline. Rather than operating as isolated chatbots, the agents are governed by a **Multi-Agent State Graph Orchestrator** that passes a single canonical state object (`ClaimState`) from First Notice of Loss (FNOL) to final payout.

To make this Multi-Agent backbone accurate and enterprise-ready, it is paired with two critical capabilities: **Agentic RAG (Retrieval-Augmented Generation)**, which grounds agent decisions in unstructured policy contracts, OEM repair tables, and historical fraud cases; and the **Model Context Protocol (MCP)**, which provides a standardized universal adapter for agents to query external databases, forensic engines, and payment APIs.

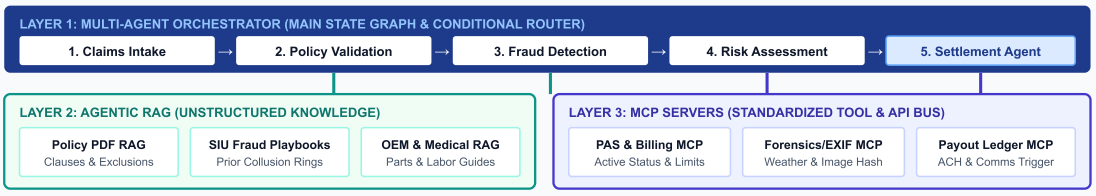


Figure 1.1 — Architectural Convergence: Multi-Agent State Graph (Control Plane), Agentic RAG (Knowledge Plane), and MCP (Integration Plane).

Target Business Impact of the Integrated Multi-Agent PoC

By combining state-driven agent handoffs with deterministic policy RAG and forensic tool verification, the architecture automates routine low-severity claims end-to-end while packaging high-risk claims into complete decision briefs for human adjusters.

< 5 Mins
FNOL-to-Settlement for STP Claims (down from 7–14 days)

> 92%
Recall on Seeded Image Tampering & Soft-Fraud Anomalies

100%
Audit Traceability Across All 5 Agent Handoffs

2. The Five-Agent Adjudication Pipeline: Roles, RAG & MCP Handoffs

Each agent in the pipeline acts as a specialized domain expert with strictly scoped read/write permissions on the shared `ClaimState` object. Rather than passing raw conversational text between agents—which introduces hallucination drift—every agent writes a schema-validated JSON block that downstream agents consume deterministically.

Agent Node	Core Responsibility	RAG Knowledge Used	MCP Tools / APIs	State Output & Handoff
1. Claims Intake Agent	Extracts entities from FNOL PDFs, invoices, and damage photos; validates completeness.	Standard FNOL Document Taxonomy & Damage Class Catalog.	`ocr-vision-mcp` (Document AI, Vision Classifier, Email Hook).	Writes `intake_data`; triggers **Policy Validation**.
2. Policy Validation Agent	Verifies active policy status on loss timestamp, deductibles, and peril coverage.	**Policy Wording RAG:** Endorsement PDFs, riders, exclusion clauses.	`core-pas-mcp` (Policy Admin DB, Billing & Premium Ledger).	Writes `policy_validation`; routes to **Fraud** or **Denial**.
3. Fraud Detection Agent	Screens for metadata tampering, duplicate photos, weather mismatches, and collusion.	**SIU Casebook RAG:** Historical fraud patterns, staged-loss rings.	`forensics-mcp` (EXIF Parser, pHash DB, Weather & Telematics API).	Writes `fraud_report` (Score 0–100); routes to **Risk** or **SIU**.
4. Risk Assessment Agent	Benchmarks repair/medical line items, determines fault split, and assigns risk tier.	**Valuation RAG:** OEM parts catalog, regional labor & CPT medical guides.	`actuarial-mcp` (Cost Estimator, Subrogation & Reserve Calculator).	Writes `risk_assessment`; hands off to **Settlement**.
5. Settlement Agent	Computes net payout (less deductible/depreciation) and executes STP or HITL brief.	Regulatory Claims Settlement Notice Templates & State Rules.	`settlement-mcp` (ACH Payment Gateway, Claims Ledger, Notification).	Writes `settlement_decision`; executes payout or Adjuster queue.

3. Conditional Routing Gates & Exception Paths ("Connecting the Dots")

1. **Gate 1 — Intake Completeness Loop (Intake → Claimant):** If the *Claims Intake Agent* computes a `completeness_score < 1.0` (e.g., missing repair estimate or unreadable VIN), the graph pauses downstream execution and automatically dispatches a targeted document request to the claimant.
2. **Gate 2 — Fast-Track Coverage Denial (Policy Validation → Settlement):** If the *Policy Validation Agent* determines the policy was lapsed on the date of loss or triggers an explicit exclusion clause via RAG, the orchestrator bypasses Fraud and Risk agents and routes directly to the *Settlement Agent* to generate a cited denial letter.
3. **Gate 3 — SIU Hard-Fraud Freeze (Fraud Detection → Human SIU):** If the *Fraud Detection Agent* calculates a composite `fraud_score ≥ 75` (e.g., photo EXIF timestamp predates policy inception or pHash matches a prior claim), automated processing halts immediately and escalates to the Special Investigations Unit.
4. **Gate 4 — Straight-Through Processing vs. Adjuster HITL (Settlement Execution):** If `risk_tier == "LOW"`, `fraud_score < 25`, and net payout is within the auto-authority limit (e.g., ≤ $5,000), the *Settlement Agent* triggers instant ACH disbursement. Otherwise, it pauses for one-click Human-in-the-

4. Architectural Deep Dive: Agentic RAG vs. Model Context Protocol (MCP)

A common architectural question when designing this PoC is: **"If Multi-Agent is the main system, is RAG involved, and do we actually need MCP?"** Both play distinct, complementary roles alongside the Multi-Agent orchestrator:

- **Yes — Agentic RAG is Essential for Unstructured Grounding:** Insurance adjudication depends heavily on unstructured text that cannot be queried via simple SQL. Our PoC embeds three specialized RAG pipelines: (1) *Policy Contract RAG* for retrieving exact endorsement exclusions and sub-limits with page citations; (2) *OEM & Medical Benchmark RAG* for validating whether billed repair hours or CPT codes align with regional standards; and (3) *SIU Fraud Pattern RAG* for matching incident narratives against known modus operandi.
- **MCP (Model Context Protocol) — Optional for Sandbox, Needed for Enterprise Scale:** In a minimal Week-1 Python demo with mocked functions, agents can use native LLM tool-calling without MCP. However, in a realistic enterprise PoC connecting to Guidewire/Duck Creek PAS, EXIF forensics, weather feeds, and payment rails, hardcoding API wrappers inside each agent creates brittle spaghetti code.

1. Multi-Agent Graph (The Brain)

Role: Owns workflow state (`ClaimState`), agent sequencing, conditional branching, and Human-in-the-Loop interrupts.
Technology: LangGraph / CrewAI.

2. Agentic RAG (The Memory)

Role: Retrieves and cites clauses from policy PDFs, OEM repair manuals, and historical SIU files using hybrid semantic + keyword search.
Technology: pgvector / Pinecone + Reranker.

3. Without MCP (Sandbox Mini-PoC)

When to use: Fast single-notebook prototypes.
Limitation: Tools are tightly coupled to agent code; no standardized auth, schema discovery, or reuse across teams.

4. With MCP (Enterprise PoC Standard)

Why it is needed: Decouples agents from backend systems. Each backend exposes an MCP Server with strict JSON schemas, audit logging, and role-based access control.

5. Recommended MCP Server Topology for the Claims PoC

How the 5 Agents Connect to Enterprise Tools via MCP

In our PoC architecture, the Multi-Agent Orchestrator acts as the **MCP Host/Client**, communicating over standard JSON-RPC with four domain-scoped **MCP Servers** (including exposing the RAG vector stores as MCP resources):

- **`claims-rag-mcp-server`:** Exposes hybrid vector search tools (`search_policy_clauses`, `lookup_oem_labor_rates`) to the Policy and Risk agents with mandatory source citation metadata.
- **`core-pas-mcp-server`:** Wraps the Policy Administration System REST/SOAP APIs (`get_policy_status`, `get_coverage_limits`) exclusively for the Policy Validation Agent.
- **`forensics-verify-mcp-server`:** Exposes image EXIF inspection, perceptual hash duplicate lookup, and historical NOAA/weather API verification to the Fraud Detection Agent.
- **`settlement-ledger-mcp-server`:** Exposes guarded financial actions (`post_claim_reserve`, `execute_ach_payout`) with strict monetary thresholds to the Settlement Agent.

6. State Machine Flow & Shared Data Contract (`ClaimState`)

The diagram below illustrates the exact state transitions, RAG retrievals, MCP tool calls, and conditional exit ramps that connect the five agents into a deterministic state machine.

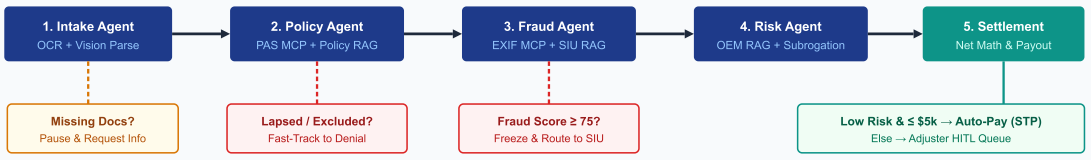


Figure 4.1 — Conditional State Routing Graph with Automated Exit Ramps for Incomplete Data, Policy Denial, SIU Freeze, and STP Disbursement.

7. Canonical JSON State Payload & Orchestrator Execution Pattern

SHARED STATE CONTRACT (CLAIMSTATE JSON)

```json
{
  "claim_id": "CLM-2026-88492",
  "intake_data": {
    "policy_no": "POL-AUTO-99201",
    "loss_type": "REAR_COLLISION",
    "claimed_amt": 3200.00,
    "completeness": 1.0
  },
  "policy_validation": {
    "active": true, "deductible": 500.0,
    "rag_clause": "Sec 4.2 Covered",
    "eligible": true
  },
  "fraud_report": {
    "fraud_score": 14,
    "exif_verified": true,
    "siu_escalate": false
  },
  "risk_assessment": {
    "oem_rag_adjusted": 2950.00,
    "depreciation": 210.00,
    "risk_tier": "LOW"
  },
  "settlement_decision": {
    "track": "STP_AUTO_PAY",
    "net_payout": 2240.00,
    "txn_ref": "ACH-7741092"
  }
}
```

LANGGRAPH + RAG + MCP ORCHESTRATOR

```python
# Define State Graph & MCP Client Bus
workflow = StateGraph(ClaimState)

workflow.add_node("intake", intake_agent)
workflow.add_node("policy", policy_rag_agent)
workflow.add_node("fraud", fraud_mcp_agent)
workflow.add_node("risk", risk_rag_agent)
workflow.add_node("settle", settlement_agent)

# Conditional Edges (Connecting Dots)
workflow.add_conditional_edges(
  "intake",
  lambda s: "policy" if s.complete else END
)
workflow.add_conditional_edges(
  "policy",
  lambda s: "fraud" if s.eligible else
"settle"
)
workflow.add_conditional_edges(
  "fraud",
  lambda s: "SIU_QUEUE" if s.score >= 75 else
"risk"
)
workflow.add_edge("risk", "settle")
app =
workflow.compile(checkpointer=postgres_cp)
```

8. Reference Technology Stack (Multi-Agent + RAG + MCP)

Architectural Layer	Recommended Stack	Role in the Insurance Claims PoC
Multi-Agent Orchestration	**LangGraph** (Python)	Manages stateful cyclical graph routing, conditional edges, state checkpointing, and Human-in-the-Loop adjuster interrupts.
Foundation LLMs	**Gemini 2.5 Pro & Flash**	Multimodal vision/PDF extraction (Intake), complex contractual reasoning (Policy & Risk), and fast structured JSON generation.
Agentic RAG Layer	**PostgreSQL (pgvector) + Hybrid BM25 Reranker**	Stores chunked Policy Endorsement PDFs, OEM Parts/Labor pricing guides, and historical SIU fraud playbooks with exact citation metadata.
Tool Connectivity (MCP)	**FastMCP / Python MCP SDK**	Wraps Core PAS API, Image EXIF/pHash Forensics, Weather API, and Payment Gateway into standardized, schema-validated MCP servers.
Vision & Forensics	**Document AI + OpenCV / ImageHash**	Parses repair invoices, verifies image EXIF timestamps/GPS coordinates, and runs perceptual hash checks against prior claim photos.
State & Audit Ledger	**PostgreSQL + FastAPI**	Persists immutable `ClaimState` transitions at every node and exposes REST webhooks for claimant portal and adjuster UI.

9. Four-Week PoC Execution Roadmap & Target KPIs

- **Week 1 — Canonical State, Synthetic Dataset & Intake Agent:** Define the `ClaimState` Pydantic schema and prepare 50 synthetic test claims (clean STP claims, lapsed policies, inflated labor invoices, and tampered EXIF photos). Build the *Claims Intake Agent* using multimodal extraction.
- **Week 2 — Policy Validation RAG & Core PAS MCP Server:** Index policy endorsement PDFs into `pgvector` with hybrid search. Build the `core-pas-mcp-server` and wire the *Policy Validation Agent* to verify coverage and fast-track excluded claims.
- **Week 3 — Fraud Forensics MCP & Risk Valuation RAG:** Implement the *Fraud Detection Agent* (EXIF metadata check, pHash duplicate check, weather verification via MCP) and the *Risk Assessment Agent* (OEM parts/labor RAG benchmarking and subrogation flagging).
- **Week 4 — Settlement Engine, End-to-End LangGraph & Shadow Testing:** Build the *Settlement Agent* (net payout calculation, STP vs. HITL routing) and run all 50 test claims in parallel with human adjusters to benchmark accuracy and latency.

> **PoC Acceptance Criteria & Sign-Off Benchmarks**
>
> The PoC will be evaluated against five quantitative thresholds during Week 4 shadow testing: **(1) Intake Precision:** ≥ 95% field extraction accuracy across FNOL PDFs and repair estimates; **(2) Policy RAG Determinism:** 100% agreement with senior adjusters on active status, deductibles, and exclusion citations; **(3) Fraud Recall:** ≥ 90% detection of seeded EXIF/duplicate anomalies with < 10% false-positive rate; **(4) Valuation Variance:** Risk Agent approved estimate within ±5% of human adjuster benchmark; **(5) STP Automation Rate:** ≥ 40% of routine low-severity claims adjudicated end-to-end in under 3 minutes.